

使用 BigQuery DataFrames 套件分析愛荷華州的酒類銷售資料

來源：<https://codelabs.developers.google.com/bigquery-dataframes-iowa-liquor-sales?hl=zh-tw>

步驟數：10

在本實驗室中，您將使用 BigQuery Studio 中的 Python 筆記本，透過 BigQuery DataFrames 清理及分析愛荷華州酒類銷售公開資料集。

目錄

1. [1. 總覽](#)
2. [2. 需求條件](#)
3. [3. 讀取公開資料集](#)
4. [4. 以圖表方式呈現資料及清理資料](#)
5. [5. 找出銷售關聯性](#)
6. [6. 比較銷售的酒類](#)
7. [7. 搭配 BigQuery DataFrames 使用 NLTK](#)
8. [8. 比較各郡的酒精飲料消費量](#)
9. [9. 清除所用資源](#)
10. [10. 恭喜！](#)

步驟 1 · 約 0 分鐘

1. 總覽

在本實驗室中，您將使用 BigQuery Studio 的 Python 筆記本中的 BigQuery DataFrames，清理及分析愛荷華州酒類銷售公開資料集。運用 BigQuery ML 和遠端函式功能，發掘洞察資料。

您將建立 Python 筆記本，比較不同地理區域的銷售情形。這項功能可調整為處理任何結構化資料。

目標

在本實驗室中，您將瞭解如何執行下列工作：

- 在 BigQuery Studio 啟用及使用 Python 筆記本
 - 使用 BigQuery DataFrames 套件連結至 BigQuery
 - 使用 BigQuery ML 建立線性迴歸
 - 使用類似 pandas 的語法執行複雜的匯總和聯結作業
-

步驟 2 · 約 0 分鐘

2. 需求條件

- [Chrome](#) 或 [Firefox](#) 瀏覽器
- 已啟用計費功能的 Google Cloud 專案

事前準備

如要按照本程式碼研究室的說明操作，您需要啟用 BigQuery Studio 的 Google Cloud 專案，並連結帳單帳戶。

1. 在 [Google Cloud 控制台](#) 的專案選擇器頁面中，選取或建立 Google Cloud [專案](#)
2. 確保您的 Google Cloud 專案有啟用計費服務。瞭解如何[檢查專案是否已啟用計費功能](#)
3. 按照操作說明[啟用 BigQuery Studio 資產管理功能](#)。

準備 BigQuery Studio

建立空白筆記本，並連線至執行階段。

1. 前往 Google Cloud 控制台的「BigQuery Studio」。
2. 按一下「+」按鈕旁的 ▼。
3. 選取「Python 筆記本」。
4. 關閉範本選取器。
5. 選取「+ Code」建立新的程式碼儲存格。
6. 從程式碼儲存格安裝最新版 BigQuery DataFrames 套件。輸入下列指令：

□□□

```
%pip install --upgrade bigframes --quiet
```

點選「執行儲存格」按鈕或按下 *Shift + Enter* 鍵，執行程式碼儲存格。

3. 讀取公開資料集

在新程式碼儲存格中執行下列指令，初始化 BigQuery DataFrames 套件：

```
import bigframes.pandas as bpd

bpd.options.bigquery.ordering_mode = "partial"
bpd.options.display.repr_mode = "deferred"
```

注意：在本教學課程中，我們使用實驗性的「部分排序模式」，搭配類似 pandas 的篩選條件時，可提高查詢效率。需要嚴格排序或索引的某些 pandas 功能可能無法運作。

使用以下指令檢查 `bigframes` 套件版本：

```
bpd.__version__
```

本教學課程需要 1.27.0 以上版本。

愛荷華州酒類零售銷售

[愛荷華州酒類零售銷售資料集](#)是透過 [Google Cloud 的公開資料集計畫](#)，在 BigQuery 上提供。這個資料集包含愛荷華州零售商自 2012 年 1 月 1 日起，向個人銷售的每筆酒類批發交易。資料由愛荷華州商務部酒精飲料部門收集。

在 BigQuery 中查詢 [bigquery-public-data.iowa_liquor_sales.sales](#)，分析愛荷華州酒類零售銷售量。使用 `bigframes.pandas.read_gbq()` 方法，從查詢字串或資料表 ID 建立 DataFrame。

在新的程式碼儲存格中執行下列程式碼，建立名為「df」的 DataFrame：

```
df = bpd.read_gbq_table("bigquery-public-data.iowa_liquor_sales.sales")
```

探索 DataFrame 的基本資訊

使用 `DataFrame.peek()` 方法下載少量資料樣本。

執行這個儲存格：

```
df.peek()
```

預期的輸出內容：

index	invoice_and_item_number	date	store_number	store_name	...
0	RINV-04620300080	2023-04-28	10197	SUNSHINE FOODS / HAWARDEN	
1	RINV-04864800097	2023-09-25	2621	HY-VEE FOOD STORE #3 / SIOUX CITY	
2	RINV-05057200028	2023-12-28	4255	FAREWAY STORES #058 / ORANGE CITY	
3	...				

注意：`head()` 需要排序，且如果您想將資料樣本視覺化，`head()` 的效率通常不如 `peek()`。

與 pandas 相同，使用 `DataFrame.dtypes` 屬性即可查看所有可用資料欄及其對應的資料類型。這些資料會以與 pandas 相容的方式公開。

執行這個儲存格：

```
df.dtypes
```

預期的輸出內容：

```
invoice_and_item_number string[pyarrow]
date      date32[day][pyarrow]
store_number      string[pyarrow]
store_name      string[pyarrow]
address string[pyarrow]
city      string[pyarrow]
zip_code      string[pyarrow]
store_location geometry
county_number string[pyarrow]
county string[pyarrow]
category      string[pyarrow]
category_name string[pyarrow]
vendor_number string[pyarrow]
vendor_name   string[pyarrow]
item_number   string[pyarrow]
item_description      string[pyarrow]
pack      Int64
bottle_volume_ml      Int64
state_bottle_cost      Float64
state_bottle_retail    Float64
bottles_sold      Int64
sale_dollars      Float64
volume_sold_liters      Float64
volume_sold_gallons      Float64

dtype: object
```

`DataFrame.describe()` 方法會查詢 `DataFrame` 中的一些基本統計資料。執行 `DataFrame.to_pandas()`，將這些摘要統計資料下載為 pandas `DataFrame`。

執行這個儲存格：

```
df.describe("all").to_pandas()
```

預期的輸出內容：

	invoice_and_item_number	date	store_number	store_name	...
nunique	30305765	<NA>	3158	3353	...
std	<NA>	<NA>	<NA>	<NA>	...
mean	<NA>	<NA>	<NA>	<NA>	...
75%	<NA>	<NA>	<NA>	<NA>	...
25%	<NA>	<NA>	<NA>	<NA>	...
count	30305765	<NA>	30305765	30305765	...
min	<NA>	<NA>	<NA>	<NA>	...
50%	<NA>	<NA>	<NA>	<NA>	...
max	<NA>	<NA>	<NA>	<NA>	...
9 rows × 24 columns					

步驟 4 · 約 0 分鐘

4. 以圖表方式呈現資料及清理資料

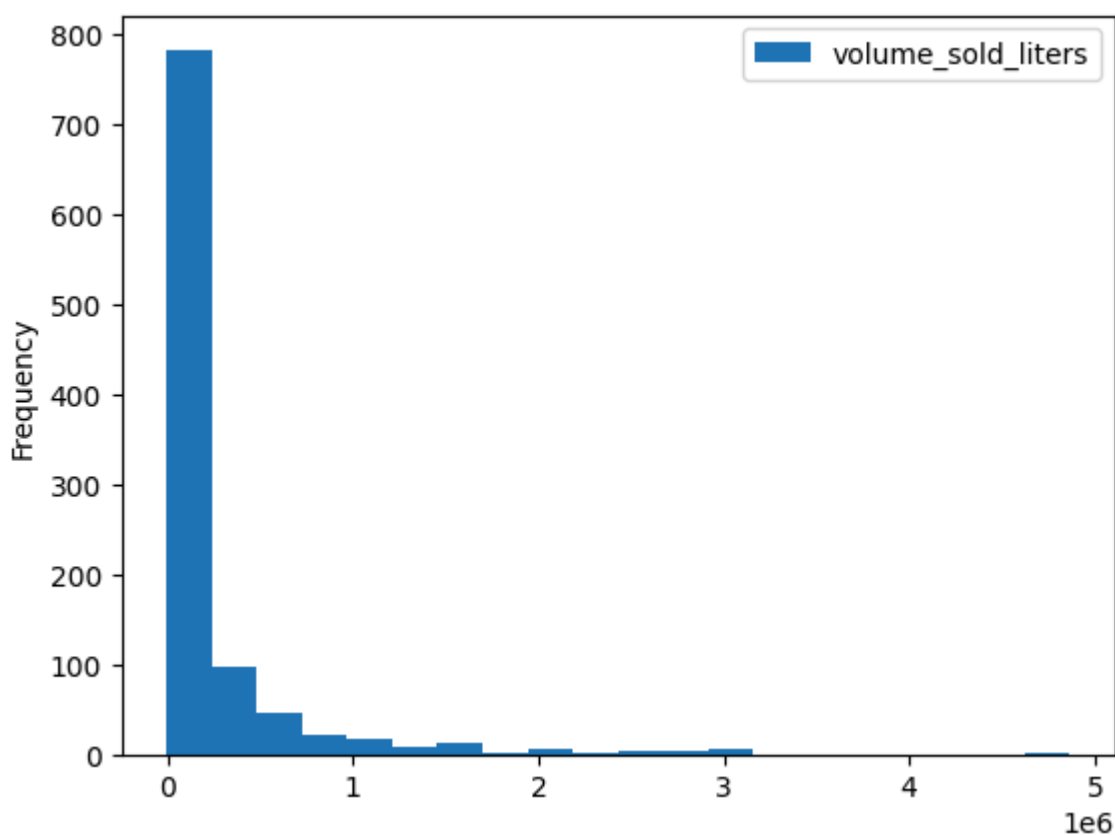
愛荷華州酒類零售銷售資料集提供精細的地理資訊，包括零售商店的位置。您可以根據這些資料找出不同地理區域的趨勢和差異。

依郵遞區號顯示銷售額

內建多種視覺化方法，例如 `DataFrame.plot.hist()`。使用這個方法比較各郵遞區號的酒類銷售量。

```
volume_by_zip = df.groupby("zip_code").agg({"volume_sold_liters": "sum"})
volume_by_zip.plot.hist(bins=20)
```

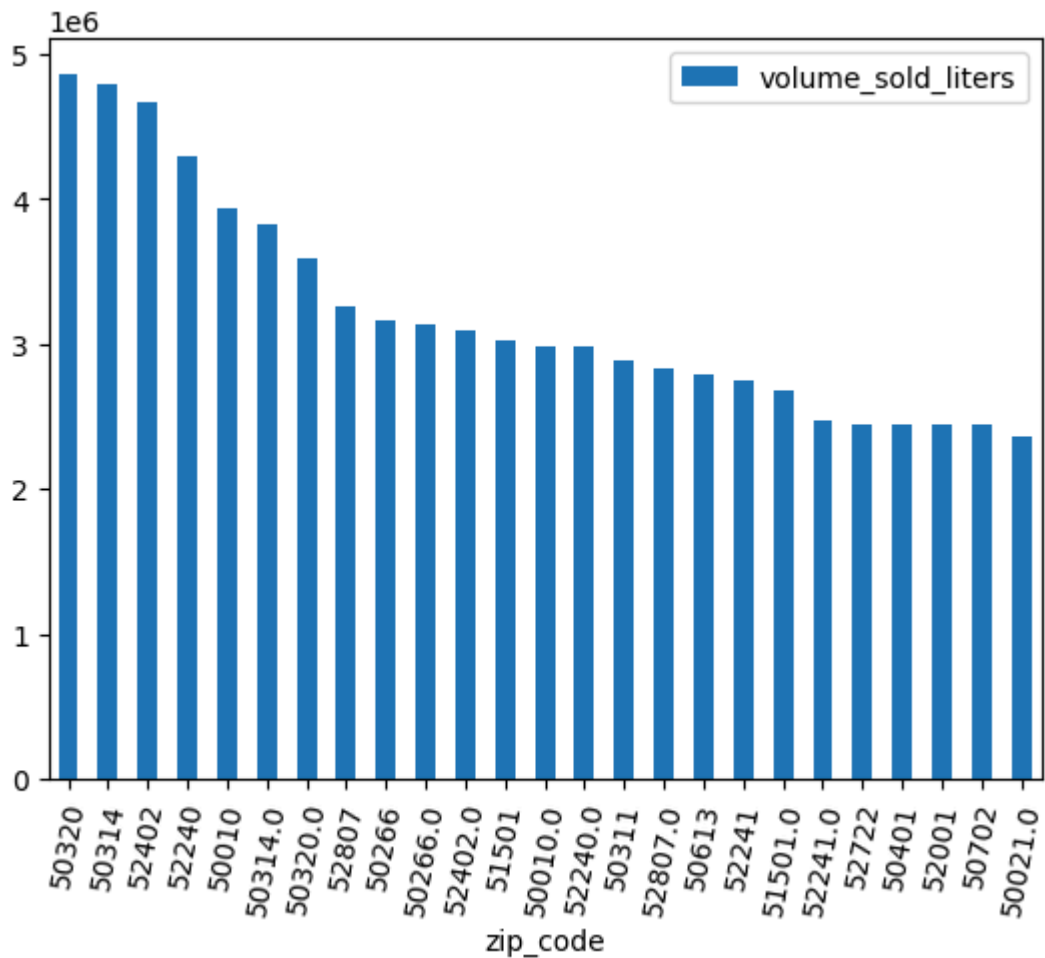
預期的輸出內容：



使用長條圖查看哪個郵遞區號的酒精飲料銷售量最高。

```
(
    volume_by_zip
    .sort_values("volume_sold_liters", ascending=False)
    .head(25)
    .to_pandas()
    .plot.bar(rot=80)
)
```

預期的輸出內容：

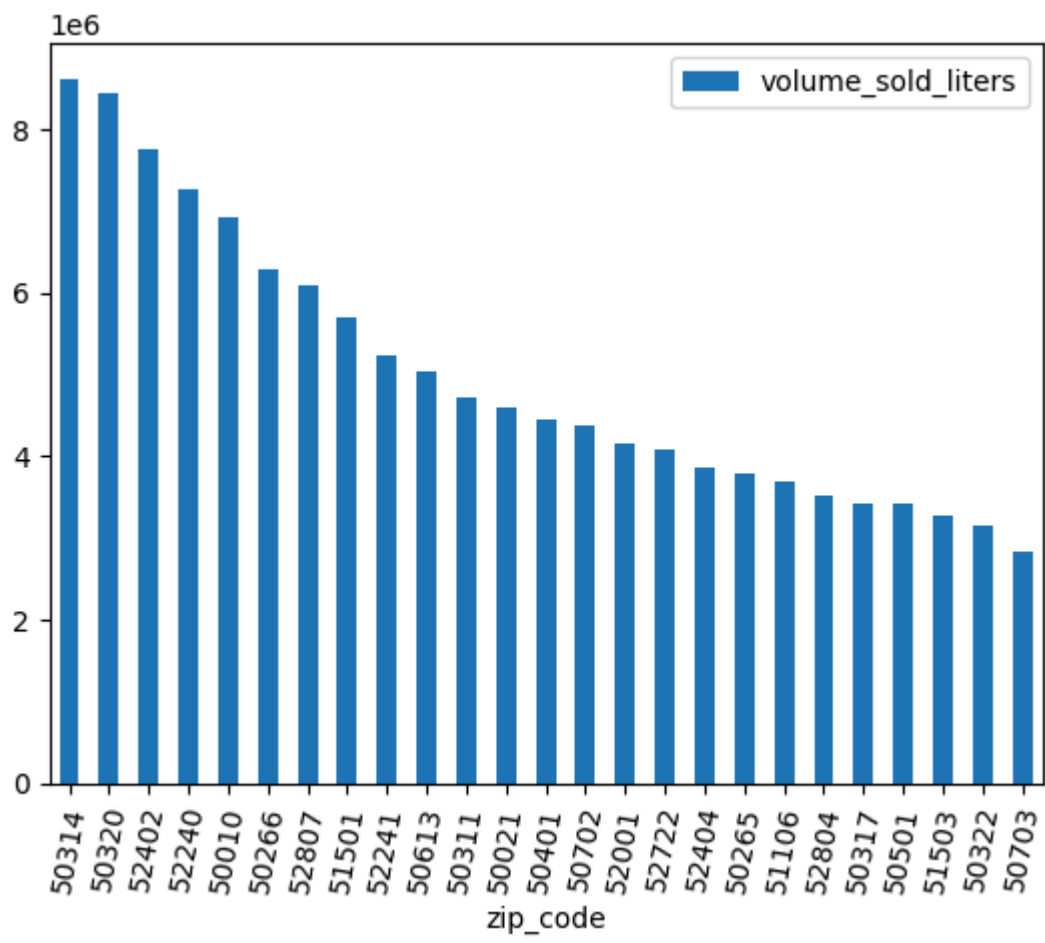


清理資料

部分郵遞區號會加上 `.0`。可能是在資料收集過程中，郵遞區號不慎轉換為浮點值。使用規則運算式清除郵遞區號，然後重複分析。

```
df = (
    bpd.read_gbq_table("bigquery-public-data.iowa_liquor_sales.sales")
    .assign(
        zip_code=lambda _: _["zip_code"].str.replace(".0", "")
    )
)
volume_by_zip = df.groupby("zip_code").agg({"volume_sold_liters": "sum"})
(
    volume_by_zip
    .sort_values("volume_sold_liters", ascending=False)
    .head(25)
    .to_pandas()
    .plot.bar(rot=80)
)
```

預期的輸出內容：



步驟 5 · 約 0 分鐘

5. 找出銷售關聯性

為什麼某些郵遞區號的銷售量比其他郵遞區號高？其中一個假設是，這是因為人口規模不同。人口較多的郵遞區號可能有較高的酒類銷售量。

計算人口數和酒類銷售量之間的相關性，藉此檢驗這項假設。

與其他資料集彙整

加入人口資料集，例如[美國人口調查局的美國社區問卷調查](#)郵遞區號製表區問卷調查。

```
census_acs = bpd.read_gbq_table("bigquery-public-data.census_bureau_acs.zcta_2020_5yr")
```

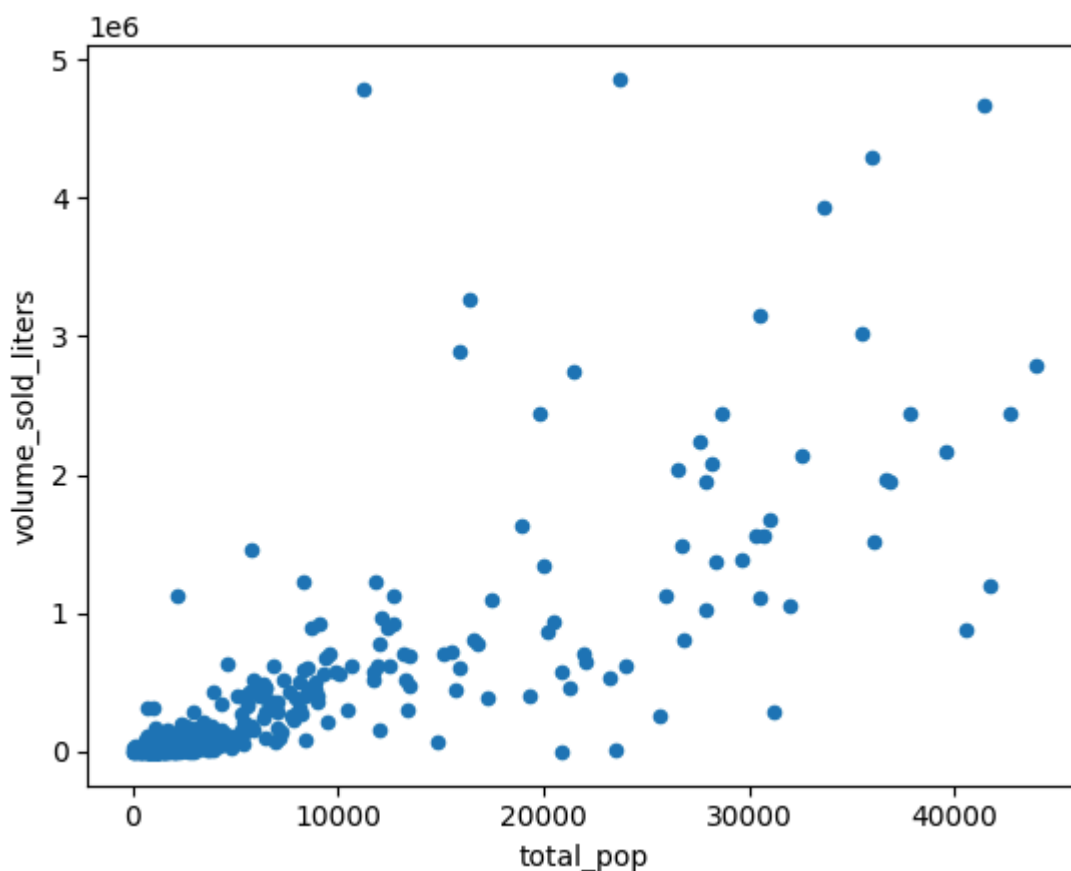
美國社區調查會以 GEOID 識別各州。如果是郵遞區號列表區域，GEOID 等於郵遞區號。

```
volume_by_pop = volume_by_zip.join(  
    census_acs.set_index("geo_id")  
)
```

建立散佈圖，比較郵遞區號列表區域人口與酒精飲料銷售量 (公升)。

```
(  
    volume_by_pop[["volume_sold_liters", "total_pop"]]  
    .to_pandas()  
    .plot.scatter(x="total_pop", y="volume_sold_liters")  
)
```

預期的輸出內容：



計算相關性

趨勢大致上呈現線性。將線性迴歸模型套用至此資料，檢查人口數預測酒類銷售額的準確度。

```
from bigframes.ml.linear_model import LinearRegression

feature_columns = volume_by_pop[["total_pop"]]
label_columns = volume_by_pop[["volume_sold_liters"]]

# Create the linear model
model = LinearRegression()
model.fit(feature_columns, label_columns)
```

使用 `score` 方法檢查模型的擬合程度。

```
model.score(feature_columns, label_columns).to_pandas()
```

輸出內容範例：

	mean_absolute_error	mean_squared_error	mean_squared_log_error
median_absolute_error		r2_score	explained_variance
0	245065.664095	224398167097.364288	5.595021
			178196.31289
	0.380096		0.380096

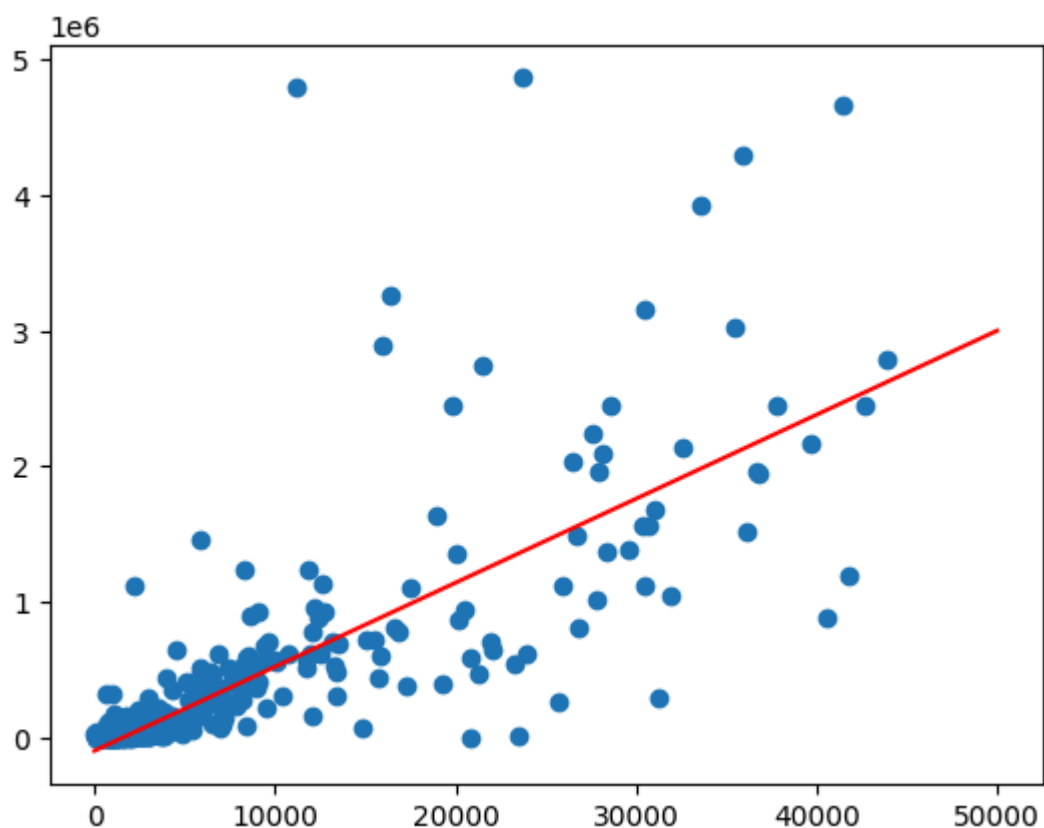
在母體值範圍呼叫 `predict` 函式，繪製最合適的線條。

```
import matplotlib.pyplot as pyplot
import numpy as np
import pandas as pd

line = pd.Series(np.arange(0, 50_000), name="total_pop")
predictions = model.predict(line).to_pandas()

zips = volume_by_pop[["volume_sold_liters", "total_pop"]].to_pandas()
pyplot.scatter(zips["total_pop"], zips["volume_sold_liters"])
pyplot.plot(
    line,
    predictions.sort_values("total_pop")["predicted_volume_sold_liters"],
    marker=None,
    color="red",
)
```

預期的輸出內容：



解決異方差問題

前一個圖表中的資料似乎具有異方差性。最佳適配線周圍的變異數會隨著母體增加。

或許每人購買的酒精飲料量相對穩定。

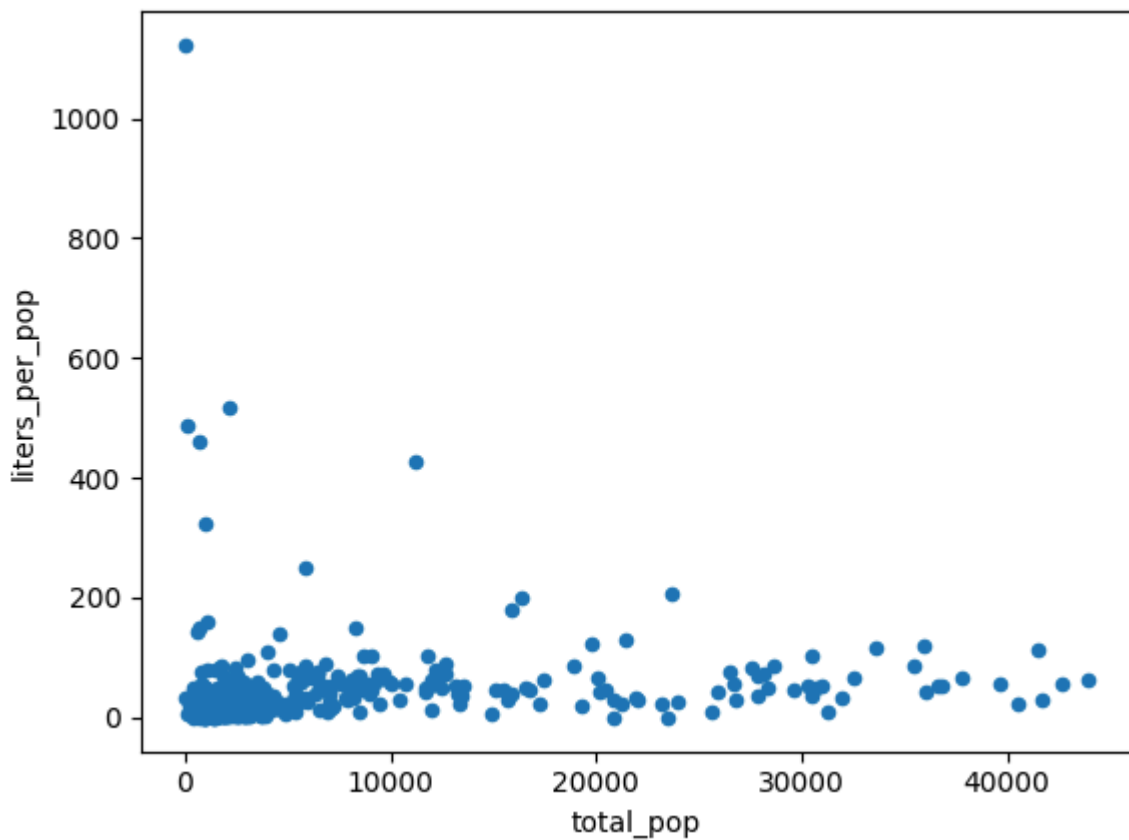
```

volume_per_pop = (
    volume_by_pop[volume_by_pop['total_pop'] > 0]
    .assign(liters_per_pop=lambda df: df["volume_sold_liters"] / df["total_pop"])
)

(
    volume_per_pop[["liters_per_pop", "total_pop"]]
    .to_pandas()
    .plot.scatter(x="total_pop", y="liters_per_pop")
)

```

預期的輸出內容：



以兩種不同方式計算購買的酒精飲料公升數平均值：

1. 在愛荷華州，每人平均購買多少酒精飲料？
2. 所有郵遞區號中，每人平均購買的酒精飲料量。

在 (1) 中，這項指標反映了整個州購買的酒精飲料量。(2) 則反映平均郵遞區號，這不一定會與 (1) 相同，因為不同郵遞區號的人口數不同。

```

df = (
    bpd.read_gbq_table("bigquery-public-data.iowa_liquor_sales.sales")
    .assign(
        zip_code=lambda _: _["zip_code"].str.replace(".", "")
    )
)

census_state = bpd.read_gbq(
    "bigquery-public-data.census_bureau_acs.state_2020_5yr",
    index_col="geo_id",
)

volume_per_pop_statewide = (
    df['volume_sold_liters'].sum()
    / census_state["total_pop"].loc['19']
)

volume_per_pop_statewide

```

預期的輸出內容： 87.997

```

average_per_zip = volume_per_pop["liters_per_pop"].mean()
average_per_zip

```

預期的輸出內容： 67.139

繪製這些平均值，與上述做法類似。

```

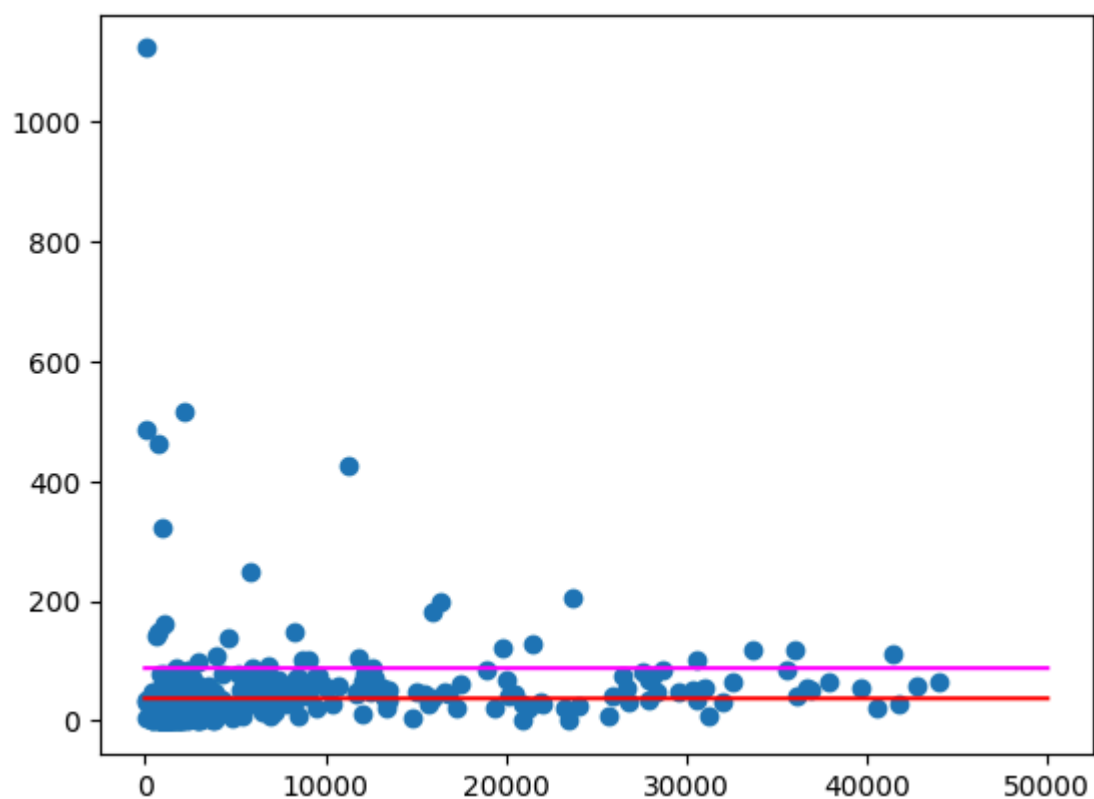
import numpy as np
import pandas as pd
from matplotlib import pyplot

line = pd.Series(np.arange(0, 50_000), name="total_pop")

zips = volume_per_pop[["liters_per_pop", "total_pop"]].to_pandas()
pyplot.scatter(zips["total_pop"], zips["liters_per_pop"])
pyplot.plot(line, np.full(line.shape, volume_per_pop_statewide), marker=None,
color="magenta")
pyplot.plot(line, np.full(line.shape, average_per_zip), marker=None, color="red")

```

預期的輸出內容：



但仍有部分郵遞區號是相當大的離群值，尤其是在人口較少的地區。請自行假設原因。舉例來說，某些郵遞區號可能人口稀少，但消費量高，因為該地區只有一家酒類商店。如果是，根據周圍郵遞區號的人口計算，甚至可以排除這些離群值。

6. 比較銷售的酒類

除了地理位置資料，愛荷華州酒類零售銷售資料庫也包含銷售商品的詳細資訊。或許透過分析這些資料，我們就能揭露不同地理區域的口味差異。

探索類別

資料庫中的項目會分類，有多少類別？

```
import bigframes.pandas as bpd

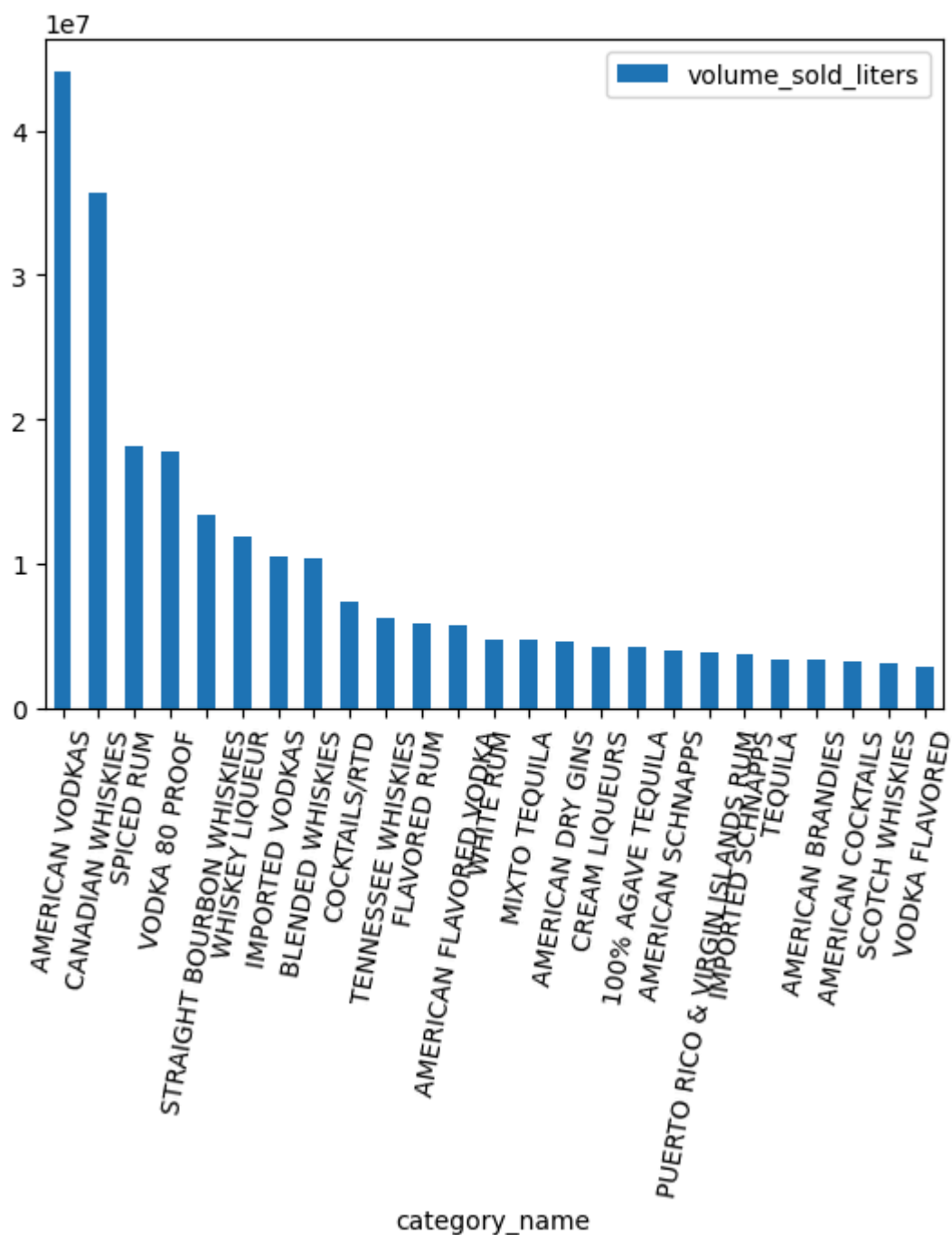
bpd.options.bigquery.ordering_mode = "partial"
bpd.options.display.repr_mode = "deferred"

df = bpd.read_gbq_table("bigquery-public-data.iowa_liquor_sales.sales")
df.category_name.nunique()
```

預期的輸出內容：`103`

哪些類別的交易量最高？

```
counts = (
    df.groupby("category_name")
    .agg({"volume_sold_liters": "sum"})
    .sort_values(["volume_sold_liters"], ascending=False)
    .to_pandas()
)
counts.head(25).plot.bar(rot=80)
```

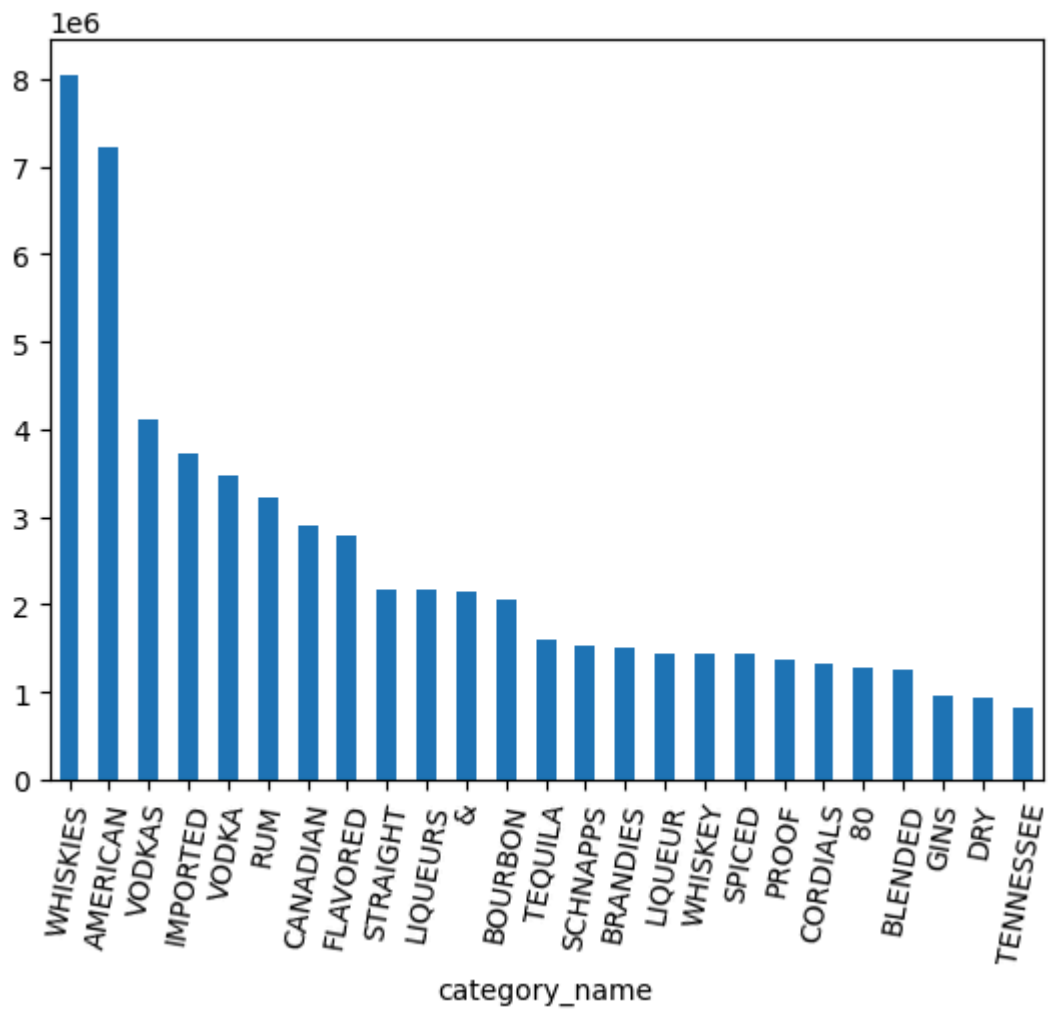


使用 ARRAY 資料類型

威士忌、蘭姆酒、伏特加等都有多個類別。我想將這些項目歸納為群組。

首先，使用 `Series.str.split()` 方法將類別名稱拆分成個別字詞。使用 `explode()` 方法，取消巢狀陣列。

```
category_parts = df.category_name.str.split(" ").explode()
counts = (
    category_parts
    .groupby(category_parts)
    .size()
    .sort_values(ascending=False)
    .to_pandas()
)
counts.head(25).plot.bar(rot=80)
```



```
category_parts.nunique()
```

預期的輸出內容： 113

如上圖所示，資料中仍有 VODKA 和 VODKAS。如要將類別摺疊成較小的集合，需要更多分組。

步驟 7 · 約 0 分鐘

7. 搭配 BigQuery DataFrames 使用 NLTK

由於只有約 100 個類別，因此可以編寫一些直觀推斷，甚至手動建立從類別到更廣泛酒類型的對應。或者，也可以使用 Gemini 等大型語言模型建立這類對應。請參閱「[使用 BigQuery DataFrames 從非結構化資料取得洞察資訊](#)」程式碼實驗室，瞭解如何搭配使用 BigQuery DataFrames 和 Gemini。

請改用較傳統的自然語言處理套件 NLTK 來處理這些資料。舉例來說，這項技術可將複數和單數名詞合併為相同的值。

使用 NLTK 詞幹還原字詞

[NLTK 套件](#)提供可從 Python 存取的自然語言處理方法。安裝套件即可試用。

```
%pip install nltk
```

接著匯入套件。檢查版本。本教學課程稍後會用到這項資訊。

```
import nltk

nltk.__version__
```

標準化字詞的方法之一是「詞幹化」。這會移除所有後置字元，例如複數的結尾「s」。

```
def stem(word: str) -> str:
    # https://www.nltk.org/howto/stem.html
    import nltk.stem.snowball

    # Avoid failure if a NULL is passed in.
    if not word:
        return word

    stemmer = nltk.stem.snowball.SnowballStemmer("english")
    return stemmer.stem(word)
```

請試著對幾個字詞使用這項功能。

```
stem("WHISKEY")
```

預期的輸出內容：`whiskey`

```
stem("WHISKIES")
```

預期的輸出內容：`whiski`

很遺憾，這並未將「whiskies」對應至「whiskey」。詞幹還原器無法處理不規則複數。請試試詞形還原器，這項工具會使用更精密的技術來找出基本字詞 (稱為「詞元」)。

```
def lemmatize(word: str) -> str:
    # https://stackoverflow.com/a/18400977/101923
    # https://www.nltk.org/api/nltk.stem.wordnet.html#module-nltk.stem.wordnet
    import nltk
    import nltk.stem.wordnet

    # Avoid failure if a NULL is passed in.
    if not word:
        return word

    nltk.download('wordnet')
    wnl = nltk.stem.wordnet.WordNetLemmatizer()
    return wnl.lemmatize(word.lower())
```

請試著對幾個字詞使用這項功能。

```
lemmatize("WHISKIES")
```

預期的輸出內容：`whisky`

```
lemmatize("WHISKY")
```

預期的輸出內容：`whisky`

```
lemmatize("WHISKEY")
```

預期的輸出內容：`whiskey`

很遺憾，這個詞形還原工具不會將「whiskey」對應至與「whiskies」相同的詞元。由於這個字詞對愛荷華州零售酒類銷售資料庫特別重要，請使用字典手動將其對應至美式拼法。

```
def lemmatize(word: str) -> str:
    # https://stackoverflow.com/a/18400977/101923
    # https://www.nltk.org/api/nltk.stem.wordnet.html#module-nltk.stem.wordnet
    import nltk
    import nltk.stem.wordnet

    # Avoid failure if a NULL is passed in.
    if not word:
        return word

    nltk.download('wordnet')
    wnl = nltk.stem.wordnet.WordNetLemmatizer()
    lemma = wnl.lemmatize(word.lower())

    table = {
        "whisky": "whiskey", # Use the American spelling.
    }
    return table.get(lemma, lemma)
```

請試著對幾個字詞使用這項功能。

```
lemmatize("WHISKIES")
```

預期的輸出內容：`whiskey`

```
lemmatize("WHISKEY")
```

預期的輸出內容：`whiskey`

恭喜！這個詞形還原工具應該很適合用來縮小類別範圍。如要搭配 BigQuery 使用，必須將其部署至雲端。

設定專案以部署函式

您需要先完成一次性設定，才能將這項函式部署至雲端，供 BigQuery 存取。

建立新的程式碼儲存格，並將 `your-project-id` 替換為您在本教學課程中使用的 Google Cloud 專案 ID。

```
project_id = "your-project-id"
```

建立不具任何權限的服務帳戶，因為這項函式不需要存取任何雲端資源。

```

from google.cloud import iam_admin_v1
from google.cloud.iam_admin_v1 import types

iam_admin_client = iam_admin_v1.IAMClient()
request = types.CreateServiceAccountRequest()

account_id = "bigframes-no-permissions"
request.account_id = account_id
request.name = f"projects/{project_id}"

display_name = "bigframes remote function (no permissions)"
service_account = types.ServiceAccount()
service_account.display_name = display_name
request.service_account = service_account

account = iam_admin_client.create_service_account(request=request)
print(account.email)

```

預期的輸出內容：`bigframes-no-permissions@your-project-id.iam.gserviceaccount.com`

建立 BigQuery 資料集來存放函式。

```

from google.cloud import bigquery

bqclient = bigquery.Client(project=project_id)
dataset = bigquery.Dataset(f"{project_id}.functions")
bqclient.create_dataset(dataset, exists_ok=True)

```

部署遠端函式

如果尚未啟用，請啟用 Cloud Functions API。

```
!gcloud services enable cloudfunctions.googleapis.com
```

現在，將函式部署到剛才建立的資料集。將 `@bpd.remote_function` 裝飾器新增至您在上一個步驟中建立的函式。

```

@bpd.remote_function(
    dataset=f"{project_id}.functions",
    name="lemmatize",
    # TODO: Replace this with your version of nltk.
    packages=["nltk==3.9.1"],
    cloud_function_service_account=f"bigframes-no-
permissions@{project_id}.iam.gserviceaccount.com",
    cloud_function_ingress_settings="internal-only",
)
def lemmatize(word: str) -> str:
    # https://stackoverflow.com/a/18400977/101923
    # https://www.nltk.org/api/nltk.stem.wordnet.html#module-nltk.stem.wordnet
    import nltk
    import nltk.stem.wordnet

    # Avoid failure if a NULL is passed in.
    if not word:
        return word

    nltk.download('wordnet')
    wnl = nltk.stem.wordnet.WordNetLemmatizer()
    lemma = wnl.lemmatize(word.lower())

    table = {
        "whisky": "whiskey", # Use the American spelling.
    }
    return table.get(lemma, lemma)

```

部署作業大約需要兩分鐘。

使用遠端功能

部署完成後，即可測試這項函式。

```

lemmatize = bpd.read_gbq_function(f"{project_id}.functions.lemmatize")

words = bpd.Series(["whiskies", "whisky", "whiskey", "vodkas", "vodka"])
words.apply(lemmatize).to_pandas()

```

預期的輸出內容：

```

0      whiskey
1      whiskey
2      whiskey
3      vodka
4      vodka

dtype: string

```


步驟 8 · 約 0 分鐘

8. 比較各郡的酒精飲料消費量

現在 `lemmatize` 函式已可使用，請用它來合併類別。

找出最能歸納類別的字詞

首先，請建立資料庫中所有類別的 `DataFrame`。

```
df = bpd.read_gbq_table("bigquery-public-data.iowa_liquor_sales.sales")

categories = (
    df['category_name']
    .groupby(df['category_name'])
    .size()
    .to_frame()
    .rename(columns={"category_name": "total_orders"})
    .reset_index(drop=False)
)
categories.to_pandas()
```

預期的輸出內容：

```
category_name  total_orders
0      100 PROOF VODKA  99124
1      100% AGAVE TEQUILA    724374
2      AGED DARK RUM    59433
3      AMARETTO - IMPORTED    102
4      AMERICAN ALCOHOL    24351
...      ...      ...
98     WATERMELON SCHNAPPS    17844
99     WHISKEY LIQUEUR  1442732
100     WHITE CREME DE CACAO    7213
101     WHITE CREME DE MENTHE    2459
102     WHITE RUM    436553
103 rows x 2 columns
```

接著，建立類別中所有字詞的 `DataFrame`，但要排除標點符號和「item」等幾個填充字詞。

```

words = (
    categories.assign(
        words=categories['category_name']
            .str.lower()
            .str.split(" ")
    )
    .assign(num_words=lambda _: _['words'].str.len())
    .explode("words")
    .rename(columns={"words": "word"})
)
words = words[
    # Remove punctuation and "item", unless it's the only word
    (words['word'].str.isalnum() & ~(words['word'].str.startswith('item')))
    | (words['num_words'] == 1)
]
words.to_pandas()

```

預期的輸出內容：

	category_name	total_orders	word	num_words
0	100 PROOF VODKA 99124	100	3	
1	100 PROOF VODKA 99124	proof	3	
2	100 PROOF VODKA 99124	vodka	3	
...	...	...	...	
252	WHITE RUM 436553	white	2	
253	WHITE RUM 436553	rum	2	

254 rows × 4 columns

請注意，在分組後進行詞形還原，可減少 Cloud Function 的負載。您可以在資料庫中數百萬列的每一列套用詞形還原函式，但這樣做會比在分組後套用函式更耗費資源，且可能需要提高配額。

```

lemmas = words.assign(lemma=lambda _: _["word"].apply(lemmatize))
lemmas.to_pandas()

```

預期的輸出內容：

	category_name	total_orders	word	num_words	lemma
0	100 PROOF VODKA 99124	100	3	100	
1	100 PROOF VODKA 99124	proof	3	proof	
2	100 PROOF VODKA 99124	vodka	3	vodka	
...	...	...	...	...	
252	WHITE RUM 436553	white	2	white	
253	WHITE RUM 436553	rum	2	rum	

254 rows × 5 columns

字詞完成詞形還原後，請選取最能歸納類別的詞元。由於類別中沒有太多功能字，因此請使用啟發式方法，如果某個字詞出現在多個其他類別中，則可能更適合做為摘要字詞 (例如威士忌)。

```

lemma_counts = (
    lemmas
    .groupby("lemma", as_index=False)
    .agg({"total_orders": "sum"})
    .rename(columns={"total_orders": "total_orders_with_lemma"})
)

categories_with_lemma_counts = lemmas.merge(lemma_counts, on="lemma")

max_lemma_count = (
    categories_with_lemma_counts
    .groupby("category_name", as_index=False)
    .agg({"total_orders_with_lemma": "max"})
    .rename(columns={"total_orders_with_lemma": "max_lemma_count"})
)

categories_with_max = categories_with_lemma_counts.merge(
    max_lemma_count,
    on="category_name"
)

categories_mapping = categories_with_max[
    categories_with_max['total_orders_with_lemma'] == categories_with_max['max_lemma_count']
].groupby("category_name", as_index=False).max()
categories_mapping.to_pandas()

```

預期的輸出內容：

	category_name	total_orders	word	num_words	lemma
total_orders_with_lemma	max_lemma_count				
0	100 PROOF VODKA	99124	vodka	3	vodka 7575769 7575769
1	100% AGAVE TEQUILA	724374	tequila	3	tequila 1601092 1601092
2	AGED DARK RUM	59433	rum	3	rum 3226633 3226633
...	...	...	...	...	...
100	WHITE CREME DE CACAO	7213	white	4	white 446225 446225
101	WHITE CREME DE MENTHE	2459	white	4	white 446225 446225
102	WHITE RUM	436553	rum	2	rum 3226633 3226633
103 rows × 7 columns					

現在每個類別都有一個摘要詞形還原，請將這些詞形還原合併至原始 DataFrame。

```

df_with_lemma = df.merge(
    categories_mapping,
    on="category_name",
    how="left"
)

df_with_lemma[df_with_lemma['category_name'].notnull()].peek()

```

預期的輸出內容：

```

      invoice_and_item_number ...      lemma      total_orders_with_lemma max_lemma_count
0      S30989000030      ...      vodka      7575769      7575769
1      S30538800106      ...      vodka      7575769      7575769
2      S30601200013      ...      vodka      7575769      7575769
3      S30527200047      ...      vodka      7575769      7575769
4      S30833600058      ...      vodka      7575769      7575769
5 rows × 30 columns

```

比較各郡

比較各郡的銷售量，找出差異。

```

county_lemma = (
    df_with_lemma
    .groupby(["county", "lemma"])
    .agg({"volume_sold_liters": "sum"})
    # Cast to an integer for more deterministic equality comparisons.
    .assign(volume_sold_int64=lambda _: _['volume_sold_liters'].astype("Int64"))
)

```

找出各郡最暢銷的產品 (詞元)。

```

county_max = (
    county_lemma
    .reset_index(drop=False)
    .groupby("county")
    .agg({"volume_sold_int64": "max"})
)

county_max_lemma = county_lemma[
    county_lemma["volume_sold_int64"] == county_max["volume_sold_int64"]
]

county_max_lemma.to_pandas()

```

預期的輸出內容：

```

      volume_sold_liters      volume_sold_int64
county lemma
SCOTT  vodka      6044393.1      6044393
APPANOOSE      whiskey 292490.44      292490
HAMILTON      whiskey 329118.92      329118
...      ...      ...      ...
WORTH      whiskey 100542.85      100542
MITCHELL      vodka 158791.94      158791
RINGGOLD      whiskey 65107.8      65107
101 rows × 2 columns

```

各郡之間的差異有多大？

```
county_max_lemma.groupby("lemma").size().to_pandas()
```

預期的輸出內容：

```
lemma
american      1
liqueur      1
vodka       15
whiskey     83

dtype: Int64
```

以銷量來看，威士忌是大多數郡最受歡迎的產品，伏特加則在 15 個郡最受歡迎。與全州最受歡迎的酒類比較。

```
total_liters = (
    df_with_lemma
    .groupby("lemma")
    .agg({"volume_sold_liters": "sum"})
    .sort_values("volume_sold_liters", ascending=False)
)
total_liters.to_pandas()
```

預期的輸出內容：

```
      volume_sold_liters
lemma
vodka      85356422.950001
whiskey    85112339.980001
rum         33891011.72
american      19994259.64
imported      14985636.61
tequila     12357782.37
cocktails/rtd  7406769.87
...
```

威士忌和伏特加的銷量幾乎相同，但伏特加在全州的銷量略高於威士忌。

比較比例

每個郡的銷售有何獨特之處？該郡與州內其他地區有何不同？

使用 [Cohen's h 測量](#)，找出酒類銷售量與全州銷售比例預期值比例差異最大的地區。

```
import numpy as np

total_proportions = total_liters / total_liters.sum()
total_phi = 2 * np.arcsin(np.sqrt(total_proportions))

county_liters = df_with_lemma.groupby(["county", "lemma"]).agg({"volume_sold_liters": "sum"})
county_totals = df_with_lemma.groupby(["county"]).agg({"volume_sold_liters": "sum"})
county_proportions = county_liters / county_totals
county_phi = 2 * np.arcsin(np.sqrt(county_proportions))

cohens_h = (
    (county_phi - total_phi)
    .rename(columns={"volume_sold_liters": "cohens_h"})
    .assign(cohens_h_int=lambda _: (_['cohens_h'] * 1_000_000).astype("Int64"))
)
```

現在已測量每個詞元的 Cohen's h，請找出各郡與全州比例的最大差異。

```
# Note: one might want to use the absolute value here if interested in counties
# that drink _less_ of a particular liquor than expected.
largest_per_county = cohens_h.groupby("county").agg({"cohens_h_int": "max"})
counties = cohens_h[cohens_h['cohens_h_int'] == largest_per_county["cohens_h_int"]]
counties.sort_values('cohens_h', ascending=False).to_pandas()
```

預期的輸出內容：

```
      cohens_h      cohens_h_int
county lemma
EL PASO liqueur 1.289667      1289667
ADAMS  whiskey 0.373591      373590
IDA    whiskey 0.306481      306481
OSCEOLA whiskey 0.295524      295523
PALO ALTO      whiskey 0.293697      293696
...      ...      ...      ...
MUSCATINE      rum      0.053757      53757
MARION  rum      0.053427      53427
MITCHELL      vodka 0.048212      48212
WEBSTER rum      0.044896      44895
CERRO GORDO      cocktails/rtd 0.027496      27495
100 rows × 2 columns
```

Cohen's h 值越大，表示該類型酒精飲料的消耗量與州平均值相比，越有可能存在統計上的顯著差異。如果正值較小，表示用量與全州平均值不同，但可能是隨機差異所致。

附註：EL PASO 郡似乎不是愛荷華州的郡，這可能表示在完全依據這些結果之前，還需要清理資料。

顯示郡

使用 [bigquery-public-data.geo_us_boundaries.counties](#) 表格加入，取得每個郡的地理區域。美國境內有許多同名的郡，因此請篩選出愛荷華州的郡。愛荷華州的 FIPS 代碼為「19」。

```

counties_geo = (
    bpd.read_gbq("bigquery-public-data.geo_us_boundaries.counties")
    .assign(county=lambda _: _['county_name'].str.upper())
)
counties_plus = (
    counties
    .reset_index(drop=False)
    .merge(counties_geo[counties_geo['state_fips_code'] == '19'], on="county", how="left")
    .dropna(subset=["county_geom"])
    .to_pandas()
)
counties_plus

```

預期的輸出內容：

```

county lemma cohens_h cohens_h_int geo_id state_fips_code ...
0 ALLAMAKEE american 0.087931 87930 19005 19 ...
1 BLACK HAWK american 0.106256 106256 19013 19 ...
2 WINNESHIEK american 0.093101 93101 19191 19 ...
... ... ... ... ... ... ... ... ...
... ... ... ... ... ... ... ... ...
96 CLINTON tequila 0.075708 75707 19045 19 ...
97 POLK tequila 0.087438 87438 19153 19 ...
98 LEE schnapps 0.064663 64663 19111 19 ...
99 rows × 23 columns

```

使用 GeoPandas 在地圖上顯示這些差異。

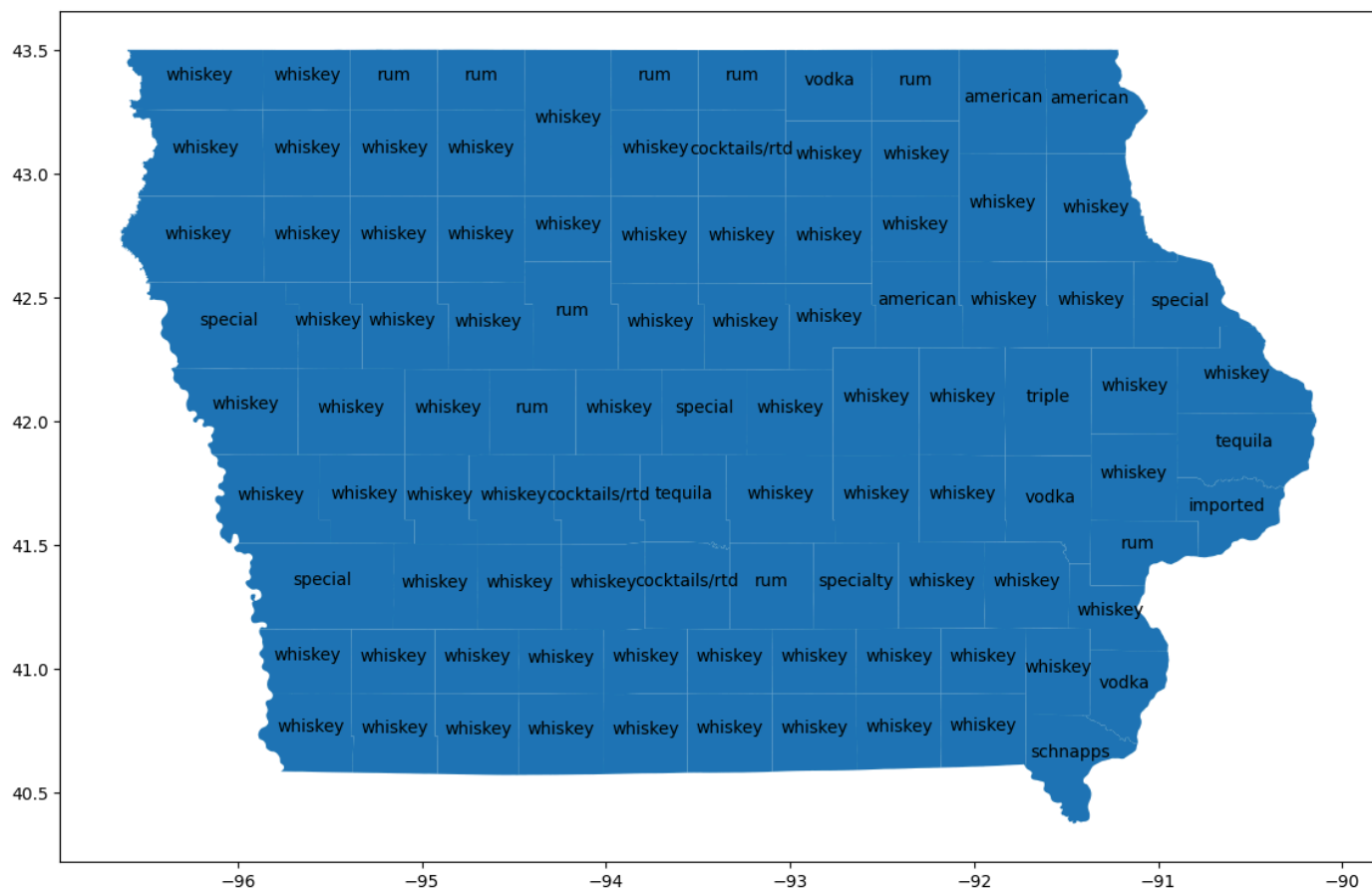
```

import geopandas

counties_plus = geopandas.GeoDataFrame(counties_plus, geometry="county_geom")

# https://stackoverflow.com/a/42214156/101923
ax = counties_plus.plot(figsize=(14, 14))
counties_plus.apply(
    lambda row: ax.annotate(
        text=row['lemma'],
        xy=row['county_geom'].centroid.coords[0],
        ha='center'
    ),
    axis=1,
)

```

步驟 9 · 約 0 分鐘

9. 清除所用資源

如果您是為了這個教學課程建立新的 Google Cloud 專案，可以[刪除專案](#)，以免系統向您收取建立資料表或其他資源的額外費用。

或者，刪除為本教學課程建立的 Cloud Functions、服務帳戶和資料集。

步驟 10 · 約 0 分鐘

10. 恭喜！

您已使用 BigQuery DataFrame 清理及分析結構化資料。您已瞭解 Google Cloud 的公開資料集、BigQuery Studio 中的 Python 筆記本、BigQuery ML、BigQuery 遠端函式，以及 BigQuery DataFrames 的強大功能。太棒了！

後續步驟

- 將這些步驟套用至其他資料，例如[美國姓名資料庫](#)。
 - 請嘗試在筆記本中生成 Python 程式碼。BigQuery Studio 中的 Python 筆記本採用 Colab Enterprise 技術。提示：要求生成測試資料相當實用。
 - 在 GitHub 上探索 [BigQuery DataFrames 的範例筆記本](#)。
 - [排定在 BigQuery Studio 中執行筆記本的時間](#)。
 - 部署[搭配 BigQuery DataFrames 的遠端函式](#)，將第三方 Python 套件與 BigQuery 整合。
-